Osquery Exploration and Endpoint Visibility using Wazuh SIEM

Real-time Endpoint Monitoring, Investigation, and Security Telemetry Collection using Osquery and Wazuh

Project Overview / Description

Osquery is an open-source endpoint visibility framework that allows security teams to query operating system data using SQL-like syntax. Instead of relying on traditional logs alone, osquery exposes live system state such as running processes, open network connections, scheduled tasks, installed software, users, and system configuration.

In this project, osquery is installed and explored at the endpoint level to understand its capabilities and use cases. The collected telemetry is then integrated with Wazuh SIEM to enable centralized monitoring, detection, and investigation. This integration enhances SOC analysts' ability to perform real-time system inspection, detect anomalies, and correlate endpoint activity with security alerts.

This project demonstrates how osquery improves endpoint security monitoring when combined with Wazuh, providing deep visibility, structured data, and continuous security intelligence.

Tools & Technologies Used

- **Osquery** – Endpoint visibility and telemetry collection
- **Wazuh SIEM** – Log ingestion, detection rules, alerting, dashboards
- **Linux (Ubuntu)** – Endpoint operating system
- **OpenSearch Dashboard (Wazuh UI)** – Visualization and analysis

Why Osquery?

Traditional endpoint monitoring relies heavily on logs that are generated only after events occur. Osquery provides **live system state** instead of passive logs.

Key strengths of Osquery:

- SQL-based querying (easy for analysts)
- Cross-platform (Linux, Windows, macOS)
- Real-time visibility
- Structured JSON output
- Ideal for threat hunting and incident response

Why Integrate Osquery with Wazuh?

When integrated with Wazuh, osquery becomes far more powerful:

- Centralized visibility of all endpoints
- Automated detection using Wazuh rules
- Correlation with other security events
- SOC dashboards for investigation
- Alerting on suspicious system behavior

This combination turns raw endpoint data into **actionable security intelligence**.

Architecture Overview

Endpoint (Osquery)

↓

Osquery JSON Logs

↓

Wazuh Agent

↓

Wazuh Manager

↓

OpenSearch / Dashboard

↓

SOC Analyst

IMPLEMENTATION STEPS

Step 1: Osquery Installation

Osquery was installed on the endpoint using the official package repository. This ensures the latest stable version with full feature support.

Purpose:
To deploy osquery on the system for endpoint visibility and querying.

```
root@client:/home/ubuntu# curl -L https://pkg.osquery.io/deb/pubkey.gpg | sudo apt-key add -
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
Warning: apt-key is deprecated. Manage keyring files in trusted.gpg.d instead (see apt-key(8)).
100  3069  100  3069    0     0  12992      0 --:--:-- --:--:-- --:--:-- 12992
OK
root@client:/home/ubuntu# sudo add-apt-repository "deb [arch=amd64] https://pkg.osquery.io/deb deb main"
Repository: 'deb [arch=amd64] https://pkg.osquery.io/deb deb main'
Description:
Archive for codename: deb components: main
More info: https://pkg.osquery.io/deb
Adding repository.
Press [ENTER] to continue or Ctrl-c to cancel.
Adding deb entry to /etc/apt/sources.list.d/archive_uri-https_pkg_osquery_io_deb-noble.list
Adding disabled deb-src entry to /etc/apt/sources.list.d/archive_uri-https_pkg_osquery_io_deb-noble.list
```

Step 2: Osquery Installation Verification

The installation was verified by checking the osquery version and launching the osquery interactive shell.

Purpose:
To confirm that osquery is installed correctly and operational.

```
root@client:/home/ubuntu# osqueryi --version
osqueryi version 5.21.0
```

Step 3: Exploring Osquery Schema and Users Table

Commands such as .schema and querying the users table were executed to explore available system tables and user accounts.

What this shows:

- Local users on the system
- User IDs, shells, and home directories

Security Use Case:
Helps detect unauthorized users, privilege escalation, or rogue accounts during investigations.

```
osquery> .schema users
CREATE TABLE users(`uid` BIGINT, `gid` BIGINT, `uid_signed` BIGINT, `gid_signed` BIGINT, `username` TEXT, `description` TEXT, `directory` TEXT, `shell` TEXT, `uuid` TEXT, `t
ype` TEXT HIDDEN, `is_hidden` INTEGER HIDDEN, `pid_with_namespace` INTEGER HIDDEN, `include_remote` INTEGER HIDDEN, PRIMARY KEY (`uid`, `username`, `uuid`, `pid_with_namespa
ce`, `include_remote`)) WITHOUT ROWID;
osquery> select * from users;
+------+------+-----------+------------+----------+-------------+----------+----------+------+
| uid  | gid  | uid_signed | gid_signed | username | description | directory | shell | uuid |
+------+------+-----------+------------+----------+-------------+----------+----------+------+
| 0    | 0    | 0         | 0          | root     | root        | /root     | /bin/bash |      |
| 1    | 1    | 1         | 1          | daemon   | daemon      | /usr/sbin | /usr/sbin/nologin |      |
| 2    | 2    | 2         | 2          | bin      | bin         | /bin      | /usr/sbin/nologin |      |
| 3    | 3    | 3         | 3          | sys      | sys         | /dev      | /usr/sbin/nologin |      |
| 4    | 65534 | 4         | 65534      | sync     | sync        | /bin      | /bin/sync |      |
| 5    | 60   | 5         | 60         | games    | games       | /usr/games | /usr/sbin/nologin |      |
| 6    | 12   | 6         | 12         | man      | man         | /var/cache/man | /usr/sbin/nologin |      |
| 7    | 7    | 7         | 7          | lp       | lp          | /var/spool/lpd | /usr/sbin/nologin |      |
| 8    | 8    | 8         | 8          | mail     | mail        | /var/mail | /usr/sbin/nologin |      |
| 9    | 9    | 9         | 9          | news     | news        | /var/spool/news | /usr/sbin/nologin |      |
| 10   | 10   | 10        | 10         | uucp     | uucp        | /var/spool/uucp | /usr/sbin/nologin |      |
| 13   | 13   | 13        | 13         | proxy    | proxy       | /bin      | /usr/sbin/nologin |      |
| 33   | 33   | 33        | 33         | www-data | www-data    | /var/www  | /usr/sbin/nologin |      |
```

Step 4: Exploring Running Processes and Listening Ports

Queries were executed to retrieve:

- Process IDs (PID)
- Listening ports
- Network protocols
- Bound addresses

Security Use Case:
Identifies suspicious services, unexpected open ports, or malware listening for connections.

```
osquery> select pid, port, protocol, address from listening_ports;
+------+------+----------+------------+
| pid  | port | protocol | address    |
+------+------+----------+------------+
| 301  | 53   | 6        | 127.0.0.54 |
| 1399 | 22   | 6        | 0.0.0.0    |
| 301  | 53   | 6        | 127.0.0.53 |
| 795  | 80   | 6        | ::         |
| 1399 | 22   | 6        | ::         |
| 301  | 53   | 17       | 127.0.0.54 |
| 301  | 53   | 17       | 127.0.0.53 |
| 462  | 68   | 17       | 10.0.1.214 |
| 696  | 323  | 17       | 127.0.0.1  |
| 696  | 323  | 17       | ::1        |
| 462  | 58   | 255      | ::         |
```

Step 5: Exploring Installed Packages and Versions

The deb packages table was queried to list installed software and versions.

Security Use Case:
Helps identify:

- Vulnerable software
- Unauthorized applications
- Software inventory for compliance

```
osquery> select name, version from deb_packages limit 10;
+--------------------------+-------------------------------------+
| name                     | version                             |
+--------------------------+-------------------------------------+
| acpid                    | 1:2.0.34-1ubuntu2                   |
| adduser                  | 3.137ubuntu1                        |
| amd64-microcode          | 3.20250311.1ubuntu0.24.04.1         |
| apache2                  | 2.4.58-1ubuntu8.10                  |
| apache2-bin              | 2.4.58-1ubuntu8.10                  |
| apache2-data             | 2.4.58-1ubuntu8.10                  |
| apache2-utils            | 2.4.58-1ubuntu8.10                  |
| apparmor                 | 4.0.1really4.0.1-0ubuntu0.24.04.5   |
| apport                   | 2.28.1-0ubuntu3.8                   |
| apport-core-dump-handler | 2.28.1-0ubuntu3.8                   |
+--------------------------+-------------------------------------+
osquery>
```

Step 6: Exploring Open Network Sockets

Queries were run to inspect active and remote network connections.

Security Use Case:
Detects outbound connections to suspicious IPs, lateral movement, or data exfiltration attempts.

```
osquery> select pid, local_address, remote_address, remote_port from process_open_sockets limit 10;
+------+---------------+----------------+-------------+
| pid  | local_address | remote_address | remote_port |
+------+---------------+----------------+-------------+
| 301  | 127.0.0.54    | 0.0.0.0        | 0           |
| 1399 | 0.0.0.0       | 0.0.0.0        | 0           |
| 301  | 127.0.0.53    | 0.0.0.0        | 0           |
| 876  | [redacted]    | [redacted]     | 1514        |
| 1813 | [redacted]    | 10.0[redacted] | 8754        |
| 795  | ::            | ::             | 0           |
| 1399 | ::            | ::             | 0           |
| 301  | 127.0.0.54    | 0.0.0.0        | 0           |
| 301  | 127.0.0.53    | 0.0.0.0        | 0           |
| 462  | [redacted]    | 0.0.0.0        | 0           |
+------+---------------+----------------+-------------+
```

Step 7: Exploring Scheduled Tasks (Cron Jobs)

The crontab table was queried to identify scheduled tasks and background jobs.

Security Use Case:
Useful for detecting persistence mechanisms such as malicious cron jobs.

```
| event | minute  | hour | day_of_month | month | day_of_week | command
| path                    |
+-------+---------+------+--------------+-------+-------------+-----------------------------------------------------------------------------------------------------------------------------------------------------------------
+-------------------------+
|       | 17      | *    | *            | *     | *           | root cd / && run-parts --report /etc/cron.hourly
| /etc/crontab            |
|       | 25      | 6    | *            | *     | *           | root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.daily; }
| /etc/crontab            |
|       | 47      | 6    | *            | *     | 7           | root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.weekly; }
| /etc/crontab            |
|       | 52      | 6    | 1            | *     | *           | root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.monthly; }
| /etc/crontab            |
|       | 30      | 3    | *            | *     | 0           | root test -e /run/systemd/system || SERVICE_MODE=1 /usr/lib/x86_64-linux-gnu/e2fsprogs/e2scrub_all_cron
| /etc/cron.d/e2scrub_all |
|       | 10      | 3    | *            | *     | *           | root test -e /run/systemd/system || SERVICE_MODE=1 /sbin/e2scrub_all -A -r
| /etc/cron.d/e2scrub_all |
|       | 09,39   | *    | *            | *     | *           | root [ -x /usr/lib/php/sessionclean ] && if [ ! -d /run/systemd/system ]; then /usr/lib/php/sessionclean; fi
| /etc/cron.d/php         |
|       | 5-55/10 | *    | *            | *     | *           | root command -v debian-sa1 > /dev/null && debian-sa1 1 1
| /etc/cron.d/sysstat     |
|       | 59      | 23   | *            | *     | *           | root command -v debian-sa1 > /dev/null && debian-sa1 60 2
| /etc/cron.d/sysstat     |
+-------+---------+------+--------------+-------+-------------+-----------------------------------------------------------------------------------------------------------------------------------------------------------------
+-------------------------+
```

Step 8: Osquery Configuration Setup

A custom osquery configuration file was created to define:

- Scheduled queries
- Logging behavior
- Enabled query packs

Purpose:
To automate data collection and ensure continuous monitoring.

```
  GNU nano 7.2                                                  /etc/osquery/osquery.conf *
{
  "options": {
    "config_plugin": "filesystem",
    "logger_plugin": "filesystem",
    "logger_path": "/var/log/osquery",
    "log_result_events": "true",
    "log_status": "true",
    "schedule_splay_percent": "10",
    "utc": "true"
  },

  "schedule": {
    "system_info": {
      "query": "SELECT hostname, cpu_brand, physical_memory FROM system_info;",
      "interval": 3600,
      "description": "System inventory"
    },

    "high_load_average": {
      "query": "SELECT period, average, '70%' AS threshold FROM load_average WHERE period = '15m' AND average > 0.7;",
      "interval": 900,
      "description": "High system load detection"
```

Step 9: Configuration Validation

Osquery daemon logs were reviewed to confirm:

- Configuration was loaded successfully
- Scheduled queries were executing

Purpose:
To ensure stability before SIEM integration.

```
           └─4502 /opt/osquery/bin/osqueryd

Feb 02 22:51:56 client osqueryd[4500]: osqueryd started [version=5.21.0]
Feb 02 22:51:56 client osqueryd[4502]: W0202 22:51:56.208532  4502 options.cpp:106] The CLI only flag --config_plugin set via config file will be ignored, please use a flag>
Feb 02 22:51:56 client osqueryd[4502]: W0202 22:51:56.208936  4502 options.cpp:106] The CLI only flag --logger_plugin set via config file will be ignored, please use a flag>
Feb 02 22:51:56 client osqueryd[4502]: I0202 22:51:56.209169  4502 eventfactory.cpp:156] Event publisher not enabled: BPFEventPublisher: Publisher disabled via configuration
Feb 02 22:51:56 client osqueryd[4502]: I0202 22:51:56.209343  4502 eventfactory.cpp:156] Event publisher not enabled: auditeventpublisher: Publisher disabled via configurat>
Feb 02 22:51:56 client osqueryd[4502]: I0202 22:51:56.209406  4502 eventfactory.cpp:156] Event publisher not enabled: inotify: Publisher disabled via configuration
Feb 02 22:51:56 client osqueryd[4502]: I0202 22:51:56.209465  4502 eventfactory.cpp:156] Event publisher not enabled: syslog: Publisher disabled via configuration
Feb 02 22:51:56 client osqueryd[4502]: I0202 22:51:56.382541  4502 eventfactory.cpp:352] The minimum events expiration timeout for hardware_events has been adjusted: 21660
Feb 02 22:51:56 client osqueryd[4502]: I0202 22:51:56.384236  4648 query.cpp:119] Storing initial results for new scheduled query: pack_ossec-rootkit_beurk_rootkit
Feb 02 22:51:56 client osqueryd[4502]: I0202 22:51:56.385080  4648 query.cpp:119] Storing initial results for new scheduled query: pack_ossec-rootkit_kenga3_rootkit
lines 1-23/23 (END)
root@client:/home/ubuntu# sudo nano /etc/osquery/osquery.conf
```

Step 10: Osquery Log Generation

Osquery results were written to osqueryd.results.log in JSON format.

Purpose:
This log file acts as the data source for Wazuh ingestion.

```
root@client:/home/ubuntu# sudo tail -f /var/log/osquery/osqueryd.results.log
{"name":"test_processes","hostIdentifier":"client","calendarTime":"Mon Feb  2 17:24:00 2026 UTC","unixTime":1770053040,"epoch":0,"counter":0,"numerics":false,"columns":{"nam
e":"systemd","pid":"1"},"action":"added"}
{"name":"test_processes","hostIdentifier":"client","calendarTime":"Mon Feb  2 17:24:00 2026 UTC","unixTime":1770053040,"epoch":0,"counter":0,"numerics":false,"columns":{"nam
e":"kworker/0:0H-events_highpri","pid":"10"},"action":"added"}
{"name":"test_processes","hostIdentifier":"client","calendarTime":"Mon Feb  2 17:24:00 2026 UTC","unixTime":1770053040,"epoch":0,"counter":0,"numerics":false,"columns":{"nam
e":"kworker/0:1-events","pid":"11"},"action":"added"}
{"name":"test_processes","hostIdentifier":"client","calendarTime":"Mon Feb  2 17:24:00 2026 UTC","unixTime":1770053040,"epoch":0,"counter":0,"numerics":false,"columns":{"nam
e":"kworker/u60:0-events_power_efficient","pid":"12"},"action":"added"}
{"name":"test_processes","hostIdentifier":"client","calendarTime":"Mon Feb  2 17:24:00 2026 UTC","unixTime":1770053040,"epoch":0,"counter":0,"numerics":false,"columns":{"nam
e":"systemd-journal","pid":"127"},"action":"added"}
{"name":"test_processes","hostIdentifier":"client","calendarTime":"Mon Feb  2 17:24:30 2026 UTC","unixTime":1770053070,"epoch":0,"counter":1,"numerics":false,"columns":{"nam
e":"kworker/u60:0-events_power_efficient","pid":"12"},"action":"removed"}
{"name":"test_processes","hostIdentifier":"client","calendarTime":"Mon Feb  2 17:24:30 2026 UTC","unixTime":1770053070,"epoch":0,"counter":1,"numerics":false,"columns":{"nam
e":"kworker/u60:0-events_unbound","pid":"12"},"action":"added"}
{"name":"pack_incident-response_arp_cache","hostIdentifier":"client","calendarTime":"Mon Feb  2 17:24:50 2026 UTC","unixTime":1770053090,"epoch":0,"counter":0,"numerics":fal
se,"columns":{"address":"10.0.1.197","interface":"enX0","mac":"0a:ff:f5:69:3e:e1","permanent":"0"},"action":"added"}
{"name":"pack_incident-response_arp_cache","hostIdentifier":"client","calendarTime":"Mon Feb  2 17:24:50 2026 UTC","unixTime":1770053090,"epoch":0,"counter":0,"numerics":fal
se,"columns":{"address":"10.0.1.1","interface":"enX0","mac":"0a:ff:c6:54:90:a9","permanent":"0"},"action":"added"}
```

Step 11: Wazuh Agent Configuration

The Wazuh agent configuration was updated to enable the osquery module and monitor the osquery results log.

Purpose:
To forward osquery telemetry to the Wazuh manager.

```
<!-- Osquery integration -->
<wodle name="osquery">
  <disabled>no</disabled>
  <run_daemon>no</run_daemon>
  <log_path>/var/log/osquery/osqueryd.results.log</log_path>
  <config_path>/etc/osquery/osquery.conf</config_path>
  <add_labels>yes</add_labels>
</wodle>
```

Step 12: Wazuh Osquery Rules Configuration

Custom and built-in Wazuh rules were enabled to parse and classify osquery events.

Purpose:
Transforms raw osquery data into structured alerts.

```
  GNU nano 7.2                                        /var/ossec/etc/rules/200200-osquery.xml *
<group name="osquery,">
 <!--
  <rule id="200220" level="1">
    <if_sid>1002</if_sid>
    <decoded_as>json</decoded_as>
    <description>Osquery Messages grouped.</description>
  </rule>
  -->
  <rule id="200221" level="3">
    <decoded_as>json</decoded_as>
    <field name="name">bpf_socket_events</field>
    <description>osquery: $(columns.path) socket started.</description>
    <options>no_full_log</options>
    <group>bpf_socket_events,</group>
  </rule>
  <rule id="200222" level="3">
    <decoded_as>json</decoded_as>
    <field name="name">list_processes_with_hash</field>
    <description>osquery: $(columns.path) process started.</description>
    <options>no_full_log</options>
    <group>list_processes_with_hash,</group>
  </rule>
```

Step 13: Osquery Data in Wazuh Dashboard

Osquery logs and alerts were successfully visualized in the Wazuh dashboard.

Analyst Benefits:

- Searchable endpoint telemetry
- Timeline analysis
- Correlation with other alerts
- Faster incident response

Skills Gained

- Endpoint visibility using osquery
- Threat hunting with SQL queries
- SIEM integration and log parsing
- Detection engineering
- SOC investigation workflows
- Dashboard-based analysis

Project Outcome

This project demonstrates how osquery enhances endpoint security monitoring when integrated with Wazuh. Analysts gain deep system visibility, real-time telemetry, and actionable insights that significantly improve detection and investigation capabilities.

Conclusion

Osquery transforms endpoints into queryable data sources, while Wazuh provides centralized detection and visibility. Together, they create a powerful SOC-ready solution for endpoint monitoring, threat detection, and incident response.